Complete C++ Learning Guide

Table of Contents

1. Introduction
2. Getting Started
3. Fundamentals
4. Control Flow
5. Functions
6. Object-Oriented Programming
7. Memory Management
8. Standard Template Library (STL)
9. Templates
10. Exception Handling
11. File I/O
12. Advanced Topics
13. Best Practices
14. Project Ideas

Introduction

C++ is a powerful, high-performance programming language that combines low-level memory manipulation with high-level abstractions. It's widely used in systems programming, game development, financial systems, robotics, and performance-critical applications.

Why C++?

- Extremely fast and efficient
- Low-level memory control
- Strong type system
- Object-oriented and generic programming
- Massive standard library (STL)
- Used in finance, gaming, and embedded systems

Note: C++ has a steeper learning curve than Python, but offers unmatched performance and control.

Getting Started

Installation

Windows:

1. Install Visual Studio Community (includes MSVC compiler)
2. Or install MinGW for GCC compiler
3. Or use online IDEs like Replit, LeetCode

Mac:

```bash
xcode-select --install  # Installs clang compiler
```

Linux:

```bash
sudo apt-get install build-essential  # Installs g++ and tools
```

Your First Program

```cpp
#include <iostream>  // Input/Output library

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

Compile and run:

```bash
g++ -o hello hello.cpp
./hello
```

Structure of a C++ Program

```cpp
```

```cpp
#include <iostream>  // Preprocessor directive - include header file

using namespace std;  // Use standard namespace

int main() {
    // Program logic here
    cout << "This is the main function" << endl;
    return 0;  // Return status code
}
```

Fundamentals

Variables and Data Types

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // Integer types
    int age = 20;
    short small_num = 100;  // 16-bit integer
    long large_num = 1000000L;  // 32 or 64-bit
    long long very_large = 9999999999LL;  // 64-bit

    // Unsigned types (only positive)
    unsigned int positive = 50;
    unsigned long ul = 1000;

    // Floating-point types
    float gpa = 3.9f;  // 32-bit (note 'f' suffix)
    double price = 19.99;  // 64-bit (more precise)
    long double very_precise = 3.14159265358979323846264338327 9L;

    // Character and boolean
    char letter = 'A';
    bool is_student = true;

    // Type information
    cout << "Size of int: " << sizeof(int) << " bytes" << endl;
    cout << "Size of double: " << sizeof(double) << " bytes" << endl;

    return 0;
}
```

Constants

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // Using const keyword
    const int MAX_STUDENTS = 100;
    const float PI = 3.14159f;

    // Using #define (preprocessor)
    #define COMPANY "Goldman Sachs"

    // constexpr (compile-time constant)
    constexpr int DAYS_IN_WEEK = 7;

    // These will cause compilation errors:
    // MAX_STUDENTS = 200;  // Error: cannot reassign const

    return 0;
}
```

Input and Output

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // Output
    cout << "Hello" << endl;  // endl flushes buffer
    cout << "World" << "\n";  // \n doesn't flush
    cout << "Value: " << 42 << endl;

    // Input
    string name;
    int age;

    cout << "Enter your name: ";
    getline(cin, name);  // Read entire line including spaces

    cout << "Enter your age: ";
    cin >> age;  // Read single value

    cout << "Hello " << name << ", you are " << age << " years old" << endl;

    // Multiple inputs
    int x, y, z;
    cout << "Enter three numbers: ";
    cin >> x >> y >> z;

    return 0;
}
```

Operators

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 3;

    // Arithmetic operators
    cout << a + b << endl;  // 13
    cout << a - b << endl;  // 7
    cout << a * b << endl;  // 30
    cout << a / b << endl;  // 3 (integer division)
    cout << a % b << endl;  // 1 (modulo)

    // Increment and decrement
    int x = 5;
    cout << x++ << endl;  // 5 (post-increment)
    cout << ++x << endl;  // 7 (pre-increment)

    // Comparison operators
    cout << (a > b) << endl;  // 1 (true)
    cout << (a == b) << endl;  // 0 (false)
    cout << (a != b) << endl;  // 1 (true)

    // Logical operators
    cout << (true && false) << endl;  // 0 (AND)
    cout << (true || false) << endl;  // 1 (OR)
    cout << (!true) << endl;  // 0 (NOT)

    // Bitwise operators
    cout << (a & b) << endl;  // Bitwise AND
    cout << (a | b) << endl;  // Bitwise OR
    cout << (a ^ b) << endl;  // Bitwise XOR
    cout << (a << 1) << endl;  // Left shift
    cout << (a >> 1) << endl;  // Right shift

    // Ternary operator
    int max_val = (a > b) ? a : b;

    // Assignment operators
    a += 5;  // a = a + 5
    a -= 3;  // a = a - 3
    a *= 2;  // a = a * 2
    a /= 4;  // a = a / 4
```

```
    return 0;
}
```

String Operations

cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    // String declaration and initialization
    string name = "Luis";
    string greeting = "Hello, " + name;  // Concatenation

    // String length
    cout << name.length() << endl;  // 4
    cout << name.size() << endl;  // Same as length()

    // Accessing characters
    cout << name[0] << endl;  // L
    cout << name.at(1) << endl;  // u (safer, checks bounds)

    // String methods
    string text = "C++ Programming";
    cout << text.substr(0, 3) << endl;  // "C++"
    cout << text.find("Program") << endl;  // Position of substring
    cout << text.replace(0, 3, "CPP") << endl;  // "CPP Programming"

    // String comparison
    cout << (name == "Luis") << endl;  // 1 (true)
    cout << name.compare("Luis") << endl;  // 0 (equal)

    // Appending
    name.append(" Garcia");  // or name += " Garcia"

    // Transforming
    for (char &c : text) {
        c = tolower(c);  // Convert to lowercase
    }

    return 0;
}
```

Control Flow

If/Else Statements

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int score = 85;

    if (score >= 90) {
        cout << "Grade A" << endl;
    }
    else if (score >= 80) {
        cout << "Grade B" << endl;
    }
    else if (score >= 70) {
        cout << "Grade C" << endl;
    }
    else {
        cout << "Grade F" << endl;
    }

    // Ternary operator
    string result = (score >= 70) ? "Pass" : "Fail";
    cout << result << endl;

    return 0;
}
```

Switch Statements

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int day = 3;
    string day_name;

    switch (day) {
        case 1:
            day_name = "Monday";
            break;  // Important! Prevents fall-through
        case 2:
            day_name = "Tuesday";
            break;
        case 3:
            day_name = "Wednesday";
            break;
        default:
            day_name = "Unknown";
    }

    cout << day_name << endl;  // Wednesday

    return 0;
}
```

For Loops

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // Traditional for loop
    for (int i = 0; i < 5; i++) {
        cout << i << " ";  // 0 1 2 3 4
    }
    cout << endl;

    // Loop with custom step
    for (int i = 0; i <= 10; i += 2) {
        cout << i << " ";  // 0 2 4 6 8 10
    }
    cout << endl;

    // Nested loops
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << "(" << i << "," << j << ") ";
        }
    }
    cout << endl;

    // Range-based for loop (C++11)
    int arr[] = {10, 20, 30, 40};
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    // For loop with strings
    string text = "C++";
    for (char c : text) {
        cout << c << " ";  // C + +
    }

    return 0;
}
```

While and Do-While Loops

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // While loop
    int count = 0;
    while (count < 5) {
        cout << count << " ";
        count++;
    }
    cout << endl;

    // Do-while loop (runs at least once)
    int num = 0;
    do {
        cout << num << " ";
        num++;
    } while (num < 5);
    cout << endl;

    // Breaking and continuing
    for (int i = 0; i < 10; i++) {
        if (i == 5) break;  // Exit loop
        if (i == 2) continue;  // Skip to next iteration
        cout << i << " ";  // 0 1 3 4
    }

    return 0;
}
```

Functions

Function Declaration and Definition

cpp

```cpp
#include <iostream>
using namespace std;

// Function declaration (forward declaration)
int add(int a, int b);
void greet(string name);

int main() {
    cout << add(5, 3) << endl;  // 8
    greet("Luis");

    return 0;
}

// Function definitions
int add(int a, int b) {
    return a + b;
}

void greet(string name) {
    cout << "Hello, " << name << "!" << endl;
}
```

Function Parameters and Return Types

cpp

```cpp
#include <iostream>
using namespace std;

// Return type can be: int, double, string, void, or custom type
int multiply(int x, int y) {
    return x * y;
}

// No return value
void print_info(string name, int age) {
    cout << name << " is " << age << " years old" << endl;
}

// Multiple parameters
double calculate_average(double a, double b, double c) {
    return (a + b + c) / 3.0;
}

// Return multiple values using reference parameters
void divide_with_remainder(int dividend, int divisor,
                           int &quotient, int &remainder) {
    quotient = dividend / divisor;
    remainder = dividend % divisor;
}

int main() {
    cout << multiply(4, 5) << endl;  // 20
    print_info("Luis", 20);

    int q, r;
    divide_with_remainder(17, 5, q, r);
    cout << q << " remainder " << r << endl;  // 3 remainder 2

    return 0;
}
```

Default Parameters

cpp

```cpp
#include <iostream>
using namespace std;

void print_message(string msg, int times = 1) {
    for (int i = 0; i < times; i++) {
        cout << msg << endl;
    }
}

int main() {
    print_message("Hello");  // Prints once
    print_message("Hello", 3);  // Prints three times

    return 0;
}
```

Function Overloading

cpp

```cpp
#include <iostream>
using namespace std;

// Same function name, different parameters
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    cout << add(5, 3) << endl;  // 8 (int version)
    cout << add(5.5, 2.3) << endl;  // 7.8 (double version)
    cout << add(1, 2, 3) << endl;  // 6 (three params version)

    return 0;
}
```

Pass by Value, Reference, and Pointer

cpp

```cpp
#include <iostream>
using namespace std;

// Pass by value (copy)
void increment_value(int x) {
    x++;  // Only affects local copy
}

// Pass by reference (reference to original)
void increment_reference(int &x) {
    x++;  // Affects original variable
}

// Pass by pointer (pointer to original)
void increment_pointer(int *x) {
    (*x)++;  // Affects original variable
}

int main() {
    int num = 5;

    increment_value(num);
    cout << num << endl;  // 5 (unchanged)

    increment_reference(num);
    cout << num << endl;  // 6 (changed)

    increment_pointer(&num);
    cout << num << endl;  // 7 (changed)

    return 0;
}
```

Inline Functions and Recursion

cpp

```cpp
#include <iostream>
using namespace std;

// Inline function (compiler replaces call with function body)
inline int square(int x) {
    return x * x;
}

// Recursion (function calling itself)
int factorial(int n) {
    if (n <= 1) return 1;  // Base case
    return n * factorial(n - 1);  // Recursive case
}

int main() {
    cout << square(5) << endl;  // 25
    cout << factorial(5) << endl;  // 120

    return 0;
}
```

Object-Oriented Programming

Classes and Objects

cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Person {
private:  // Private members
    string name;
    int age;

public:  // Public members
    // Constructor (initializes object)
    Person(string n, int a) {
        name = n;
        age = a;
    }

    // Getter methods
    string get_name() const {
        return name;
    }

    int get_age() const {
        return age;
    }

    // Setter methods
    void set_age(int a) {
        if (a > 0) age = a;
    }

    // Member function
    void introduce() {
        cout << "Hello, I'm " << name << " and I'm " << age << " years old" <<
    }
};

int main() {
    // Creating objects
    Person p1("Luis", 20);
    Person p2("Ana", 19);

    p1.introduce();  // Hello, I'm Luis and I'm 20 years old
    cout << p1.get_name() << endl;  // Luis
```

```cpp
    p1.set_age(21);
    cout << p1.get_age() << endl;  // 21

    return 0;
}
```

Constructors and Destructors

cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string owner;
    double balance;

public:
    // Default constructor
    BankAccount() {
        owner = "Unknown";
        balance = 0.0;
        cout << "Account created" << endl;
    }

    // Parameterized constructor
    BankAccount(string o, double b) {
        owner = o;
        balance = b;
        cout << "Account for " << owner << " created with balance $" << balance
    }

    // Copy constructor
    BankAccount(const BankAccount &other) {
        owner = other.owner;
        balance = other.balance;
        cout << "Account copied" << endl;
    }

    // Destructor (called when object is destroyed)
    ~BankAccount() {
        cout << "Account for " << owner << " closed" << endl;
    }

    void deposit(double amount) {
        balance += amount;
        cout << "Deposited $" << amount << ". New balance: $" << balance << end
    }

    void withdraw(double amount) {
        if (amount <= balance) {
```

```
            balance -= amount;
            cout << "Withdrew $" << amount << ". New balance: $" << balance <<
        }
    }
};

int main() {
    BankAccount acc1("Luis", 1000);  // Constructor called
    acc1.deposit(500);
    acc1.withdraw(200);

    return 0;  // Destructor called
}
```

Inheritance

cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

// Base class
class Animal {
protected:  // Accessible in derived classes
    string name;

public:
    Animal(string n) {
        name = n;
    }

    virtual void make_sound() {
        cout << name << " makes a sound" << endl;
    }

    void eat() {
        cout << name << " is eating" << endl;
    }
};

// Derived class
class Dog : public Animal {
private:
    string breed;

public:
    Dog(string n, string b) : Animal(n) {  // Call base constructor
        breed = b;
    }

    // Override virtual function
    void make_sound() override {
        cout << name << " barks: Woof!" << endl;
    }

    void fetch() {
        cout << name << " is fetching!" << endl;
    }
};
```

```cpp
class Cat : public Animal {
public:
    Cat(string n) : Animal(n) {}

    void make_sound() override {
        cout << name << " meows: Meow!" << endl;
    }
};

int main() {
    Dog dog("Buddy", "Golden Retriever");
    dog.make_sound();  // Buddy barks: Woof!
    dog.eat();  // Buddy is eating
    dog.fetch();  // Buddy is fetching!

    Cat cat("Whiskers");
    cat.make_sound();  // Whiskers meows: Meow!

    return 0;
}
```

Polymorphism and Virtual Functions

cpp

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    // Virtual function (can be overridden)
    virtual void draw() {
        cout << "Drawing a shape" << endl;
    }

    // Virtual destructor (important!)
    virtual ~Shape() {}
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing a rectangle" << endl;
    }
};

int main() {
    // Polymorphism: one interface, multiple implementations
    Shape *shapes[3];

    Circle c;
    Rectangle r;
    Shape s;

    shapes[0] = &c;
    shapes[1] = &r;
    shapes[2] = &s;

    for (int i = 0; i < 3; i++) {
        shapes[i]->draw();
        // Output:
```

```cpp
        // Drawing a circle
        // Drawing a rectangle
        // Drawing a shape
    }

    return 0;
}
```

Static Members

cpp

```cpp
#include <iostream>
using namespace std;

class BankAccount {
private:
    static int account_count;  // Shared by all instances
    int account_id;
    double balance;

public:
    BankAccount(double b) {
        account_count++;
        account_id = account_count;
        balance = b;
    }

    static int get_account_count() {
        return account_count;
    }

    int get_account_id() {
        return account_id;
    }
};

// Initialize static member outside the class
int BankAccount::account_count = 0;

int main() {
    BankAccount acc1(1000);
    BankAccount acc2(2000);
    BankAccount acc3(1500);

    cout << "Total accounts: " << BankAccount::get_account_count() << endl;  //
    cout << "Account ID: " << acc1.get_account_id() << endl;  // 1

    return 0;
}
```

Memory Management

Pointers

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // Declaring pointers
    int x = 10;
    int *ptr = &x;  // & is address-of operator

    // Dereferencing (accessing value at pointer)
    cout << *ptr << endl;  // 10
    cout << &x << endl;  // Memory address
    cout << ptr << endl;  // Same memory address

    // Modifying through pointer
    *ptr = 20;
    cout << x << endl;  // 20

    // Null pointer
    int *null_ptr = nullptr;  // or NULL
    if (null_ptr != nullptr) {
        cout << *null_ptr << endl;  // Safe check before dereferencing
    }

    // Pointer to pointer
    int **ptr_ptr = &ptr;
    cout << **ptr_ptr << endl;  // 20

    return 0;
}
```

Dynamic Memory Allocation

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // Allocate single integer on heap
    int *ptr = new int;
    *ptr = 42;
    cout << *ptr << endl;
    delete ptr;  // Must delete to free memory
    ptr = nullptr;  // Good practice: set to null

    // Allocate and initialize
    int *num = new int(100);
    delete num;

    // Allocate array
    int *arr = new int[5];
    for (int i = 0; i < 5; i++) {
        arr[i] = i * 10;
    }
    delete[] arr;  // Note: [] for arrays

    // Allocate object
    class Student {
    public:
        string name;
        Student(string n) { name = n; }
    };

    Student *student = new Student("Luis");
    cout << student->name << endl;  // -> for pointer to object
    delete student;

    return 0;
}
```

Smart Pointers (Automatic Memory Management)

cpp

```cpp
#include <iostream>
#include <memory>
using namespace std;

class Person {
public:
    string name;
    Person(string n) { name = n; cout << name << " created" << endl; }
    ~Person() { cout << name << " destroyed" << endl; }
};

int main() {
    // unique_ptr: exclusive ownership
    {
        unique_ptr<Person> p1(new Person("Luis"));
        // Or: auto p1 = make_unique<Person>("Luis");
        cout << p1->name << endl;
    }  // p1 automatically deleted here

    // shared_ptr: shared ownership
    {
        shared_ptr<Person> p2 = make_shared<Person>("Ana");
        shared_ptr<Person> p3 = p2;  // Both point to same object
        cout << "Reference count: " << p2.use_count() << endl;  // 2
    }  // Object deleted when all references are gone

    return 0;
}
```

References

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    // References are like const pointers
    int x = 10;
    int &ref = x;  // ref is a reference to x

    cout << ref << endl;  // 10

    ref = 20;  // Modifies x
    cout << x << endl;  // 20

    // References in functions
    void increment(int &n) {
        n++;
    }

    increment(x);
    cout << x << endl;  // 21

    // Difference from pointers
    // - References cannot be null
    // - References cannot be reassigned
    // - No * or & needed to use

    return 0;
}
```

Standard Template Library (STL)

Vectors (Dynamic Arrays)

```cpp
```

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Creating vectors
    vector<int> numbers;
    vector<string> names = {"Luis", "Ana", "Bob"};
    vector<double> prices(5);  // Vector of size 5

    // Adding elements
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    // Accessing elements
    cout << numbers[0] << endl;  // 10 (no bounds checking)
    cout << numbers.at(1) << endl;  // 20 (with bounds checking)

    // Vector information
    cout << numbers.size() << endl;  // 3
    cout << numbers.capacity() << endl;  // May be > size

    // Modifying
    numbers[0] = 15;
    numbers.insert(numbers.begin() + 1, 25);  // Insert at position
    numbers.erase(numbers.begin());  // Remove first element
    numbers.pop_back();  // Remove last element

    // Iterating
    for (int num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    // Iterator
    for (auto it = numbers.begin(); it != numbers.end(); it++) {
        cout << *it << " ";
    }

    // Clear
    numbers.clear();
```

```
    return 0;
}
```

Maps (Key-Value Pairs)

cpp

```cpp
#include <iostream>
#include <map>
using namespace std;

int main() {
    // Creating map
    map<string, int> ages;

    // Adding elements
    ages["Luis"] = 20;
    ages["Ana"] = 19;
    ages.insert({"Bob", 22});

    // Accessing
    cout << ages["Luis"] << endl;  // 20
    cout << ages.at("Ana") << endl;  // 19

    // Checking if key exists
    if (ages.find("Bob") != ages.end()) {
        cout << "Bob found" << endl;
    }

    // Iterating
    for (auto &pair : ages) {
        cout << pair.first << ": " << pair.second << endl;
    }

    // Size and clearing
    cout << ages.size() << endl;  // 3
    ages.erase("Ana");  // Remove by key

    return 0;
}
```

Sets (Unique Elements)

cpp

```cpp
#include <iostream>
#include <set>
using namespace std;

int main() {
    // Creating set
    set<int> numbers;

    // Adding elements
    numbers.insert(5);
    numbers.insert(3);
    numbers.insert(7);
    numbers.insert(3);  // Duplicate, won't be added

    cout << numbers.size() << endl;  // 3 (unique elements)

    // Iterating (automatically sorted)
    for (int num : numbers) {
        cout << num << " ";  // 3 5 7
    }
    cout << endl;

    // Checking membership
    if (numbers.count(5)) {  // or numbers.find()
        cout << "5 is in the set" << endl;
    }

    // Removing
    numbers.erase(5);

    return 0;
}
```

Strings (Advanced)

cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "C++ Programming";

    // Length and capacity
    cout << text.length() << endl;  // 15
    cout << text.capacity() << endl;

    // Substrings
    cout << text.substr(0, 3) << endl;  // "C++"
    cout << text.substr(4) << endl;  // "Programming"

    // Finding
    int pos = text.find("Program");  // 4
    if (pos != string::npos) {
        cout << "Found at position " << pos << endl;
    }

    // Replacing
    text.replace(0, 3, "CPP");  // "CPP Programming"

    // Appending and inserting
    text += "!";  // "CPP Programming!"
    text.insert(3, "-");  // "CPP- Programming!"

    // Erase
    text.erase(3, 1);  // Remove character at position 3

    return 0;
}
```

Algorithms

cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> nums = {5, 2, 8, 1, 9, 3};

    // Sorting
    sort(nums.begin(), nums.end());
    // nums is now [1, 2, 3, 5, 8, 9]

    // Reverse
    reverse(nums.begin(), nums.end());

    // Finding
    auto it = find(nums.begin(), nums.end(), 5);
    if (it != nums.end()) {
        cout << "Found 5" << endl;
    }

    // Finding with condition
    auto it2 = find_if(nums.begin(), nums.end(),
                       [](int x) { return x > 5; });

    // Counting
    int count = count(nums.begin(), nums.end(), 5);

    // Transform (apply function to all elements)
    vector<int> squared(nums.size());
    transform(nums.begin(), nums.end(), squared.begin(),
              [](int x) { return x * x; });

    // For each
    for_each(nums.begin(), nums.end(),
             [](int x) { cout << x << " "; });
    cout << endl;

    return 0;
}
```

Templates

Function Templates

cpp

```cpp
#include <iostream>
using namespace std;

// Generic function template
template <typename T>
T add(T a, T b) {
    return a + b;
}

// Multiple template parameters
template <typename T, typename U>
void print_pair(T first, U second) {
    cout << first << ", " << second << endl;
}

int main() {
    cout << add(5, 3) << endl;  // 8 (int)
    cout << add(5.5, 2.3) << endl;  // 7.8 (double)
    cout << add("Hello", " World") << endl;  // "Hello World" (string)

    print_pair(10, "test");  // 10, test
    print_pair("age", 20);  // age, 20

    return 0;
}
```

Class Templates

cpp

```cpp
#include <iostream>
using namespace std;

template <typename T>
class Stack {
private:
    vector<T> elements;

public:
    void push(T value) {
        elements.push_back(value);
    }

    T pop() {
        T value = elements.back();
        elements.pop_back();
        return value;
    }

    bool empty() {
        return elements.empty();
    }

    int size() {
        return elements.size();
    }
};

int main() {
    Stack<int> int_stack;
    int_stack.push(10);
    int_stack.push(20);
    cout << int_stack.pop() << endl;  // 20

    Stack<string> str_stack;
    str_stack.push("Hello");
    str_stack.push("World");
    cout << str_stack.pop() << endl;  // World

    return 0;
}
```

Template Specialization

cpp

```cpp
#include <iostream>
using namespace std;

// Generic template
template <typename T>
void print(T value) {
    cout << "Generic: " << value << endl;
}

// Template specialization for bool
template <>
void print<bool>(bool value) {
    cout << (value ? "True" : "False") << endl;
}

// Template specialization for string
template <>
void print<string>(string value) {
    cout << "String: " << value << endl;
}

int main() {
    print(42);  // Generic: 42
    print(true);  // True
    print("Hello");  // String: Hello

    return 0;
}
```

Exception Handling

Try-Catch Blocks

cpp

```cpp
#include <iostream>
#include <stdexcept>
using namespace std;

int main() {
    // Basic try-catch
    try {
        int x = 10, y = 0;
        if (y == 0) {
            throw invalid_argument("Division by zero!");
        }
        cout << x / y << endl;
    }
    catch (invalid_argument &e) {
        cout << "Error: " << e.what() << endl;
    }
    catch (exception &e) {
        cout << "General error: " << e.what() << endl;
    }
    catch (...) {
        cout << "Unknown error" << endl;
    }

    return 0;
}
```

Custom Exceptions

cpp

```cpp
#include <iostream>
#include <exception>
using namespace std;

// Custom exception class
class InsufficientFundsException : public exception {
public:
    const char* what() const noexcept override {
        return "Insufficient funds!";
    }
};

class BankAccount {
private:
    double balance;

public:
    BankAccount(double b) : balance(b) {}

    void withdraw(double amount) {
        if (amount > balance) {
            throw InsufficientFundsException();
        }
        balance -= amount;
    }
};

int main() {
    BankAccount acc(100);

    try {
        acc.withdraw(150);
    }
    catch (InsufficientFundsException &e) {
        cout << e.what() << endl;
    }

    return 0;
}
```

File I/O

Reading and Writing Files

cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    // Writing to file
    ofstream outfile("data.txt");

    if (!outfile.is_open()) {
        cout << "Failed to open file" << endl;
        return 1;
    }

    outfile << "Luis Garcia" << endl;
    outfile << "Age: 20" << endl;
    outfile << "GPA: 3.9" << endl;
    outfile.close();

    // Reading from file
    ifstream infile("data.txt");
    string line;

    while (getline(infile, line)) {
        cout << line << endl;
    }
    infile.close();

    return 0;
}
```

Reading Structured Data

cpp

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
using namespace std;

struct Student {
    string name;
    int age;
    double gpa;
};

int main() {
    vector<Student> students;
    ifstream file("students.txt");
    string line;

    while (getline(file, line)) {
        stringstream ss(line);
        Student s;

        getline(ss, s.name, ',');
        ss >> s.age;
        ss.ignore();
        ss >> s.gpa;

        students.push_back(s);
    }

    for (auto &s : students) {
        cout << s.name << " - Age: " << s.age << " - GPA: " << s.gpa << endl;
    }

    return 0;
}
```

Advanced Topics

Lambdas (Anonymous Functions)

cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> nums = {1, 2, 3, 4, 5};

    // Simple lambda
    auto square = [](int x) { return x * x; };
    cout << square(5) << endl;  // 25

    // Lambda with captures
    int multiplier = 10;
    auto multiply = [multiplier](int x) { return x * multiplier; };
    cout << multiply(5) << endl;  // 50

    // Capture by value [=] vs reference [&]
    int total = 0;
    for_each(nums.begin(), nums.end(),
             [&total](int x) { total += x; });
    cout << "Sum: " << total << endl;  // 15

    // Using lambda with sort
    sort(nums.begin(), nums.end(),
         [](int a, int b) { return a > b; });  // Descending

    return 0;
}
```

Move Semantics and Rvalue References

cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

class MyVector {
private:
    int *data;
    int size;

public:
    // Copy constructor
    MyVector(const MyVector &other) {
        cout << "Copy constructor" << endl;
        size = other.size;
        data = new int[size];
        for (int i = 0; i < size; i++) {
            data[i] = other.data[i];
        }
    }

    // Move constructor (efficient)
    MyVector(MyVector &&other) noexcept {
        cout << "Move constructor" << endl;
        size = other.size;
        data = other.data;
        other.data = nullptr;
    }
};

int main() {
    MyVector v1(v2);  // Copy constructor
    MyVector v3(MyVector());  // Move constructor

    return 0;
}
```

Namespaces

```
cpp
```

```cpp
#include <iostream>
using namespace std;

namespace Math {
    double PI = 3.14159;

    double circle_area(double radius) {
        return PI * radius * radius;
    }
}

namespace Physics {
    double PI = 3.14159;  // Different PI

    double spring_force(double k, double x) {
        return -k * x;
    }
}

int main() {
    cout << Math::PI << endl;  // 3.14159
    cout << Math::circle_area(5) << endl;

    cout << Physics::spring_force(10, 2) << endl;  // -20

    return 0;
}
```

Best Practices

Code Style

cpp

```cpp
// Use meaningful names
int student_count = 50;  // Good
int sc = 50;  // Bad

// Constants in UPPER_CASE
const double TAX_RATE = 0.08;

// Use braces even for single statements
if (x > 0) {
    cout << "Positive" << endl;
}

// Proper indentation (2 or 4 spaces)
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        cout << i << endl;
    }
}
```

Memory Safety

cpp

```cpp
// Always initialize pointers
int *ptr = nullptr;  // Good
int *ptr;  // Bad (uninitialized)

// Use RAII (Resource Acquisition Is Initialization)
{
    ofstream file("data.txt");
    file << "Data";
}  // File automatically closed

// Prefer smart pointers
unique_ptr<MyClass> obj(new MyClass());
// delete not needed

// Check for null before dereferencing
if (ptr != nullptr) {
    cout << *ptr << endl;
}
```

Performance Tips

```cpp
// Use references to avoid copying
void process(const vector<int> &data) {  // Better
    // ...
}

void process(vector<int> data) {  // Avoid (copies entire vector)
    // ...
}

// Use move semantics when appropriate
MyClass obj = expensive_operation();  // Uses move, not copy

// Reserve space for vectors if size is known
vector<int> nums;
nums.reserve(1000);  // Avoid reallocations

// Use const correctly
const int MAX = 100;  // Constant value
const int *ptr;  // Pointer to constant
int * const ptr;  // Constant pointer
const int * const ptr;  // Both constant
```

Project Ideas

Beginner Projects

1. **Calculator**: Basic arithmetic with operator overloading
2. **String Manipulation**: Reverse, palindrome checker, anagram detector
3. **Number Guessing Game**: Simple game with user input
4. **Temperature Converter**: Between Celsius, Fahrenheit, Kelvin
5. **Matrix Operations**: Addition, multiplication, transpose

Intermediate Projects

1. **Bank Account System**: Classes, inheritance, file I/O
2. **Student Management**: Vector of objects, sorting, searching
3. **Linked List Implementation**: Data structure from scratch

4. **Binary Search Tree**: Tree traversal, insertion, deletion
5. **File Analyzer**: Read file, count words/lines, find patterns

Advanced Projects

1. **Graph Implementation**: DFS, BFS, shortest path algorithms
2. **Stock Market Simulator**: Financial calculations and modeling
3. **Web Server**: HTTP requests, sockets, threading
4. **Game Engine**: Graphics, collision detection, game loop
5. **C++ Compiler**: Lexical analysis, parsing, code generation

---

Quick Reference

Common Libraries

Algorithms: `<algorithm>` - sort, find, transform, etc. **Containers**: `<vector>`, `<map>`, `<set>`, `<deque>` **Strings**: `<string>`, `<sstream>` **Input/Output**: `<iostream>`, `<fstream>` **Memory**: `<memory>` - smart pointers **Math**: `<cmath>` - sqrt, sin, cos, etc. **Utilities**: `<utility>`, `<functional>`

Useful Commands

bash

```
# Compile
g++ -std=c++17 -o program program.cpp

# Compile with optimizations
g++ -O2 -std=c++17 -o program program.cpp

# Compile with debugging info
g++ -g -std=c++17 -o program program.cpp

# Run
./program

# Check for memory leaks (Linux)
valgrind ./program
```

Learning Path

1. **Week 1-2**: Fundamentals, variables, control flow
2. **Week 3-4**: Functions, arrays, strings
3. **Week 5-6**: Classes, constructors, basic OOP
4. **Week 7-8**: Inheritance, polymorphism
5. **Week 9-10**: Pointers, dynamic memory
6. **Week 11-12**: STL containers and algorithms
7. **Week 13-14**: Templates, advanced OOP
8. **Week 15+**: Projects and specialization

Resources

- **Official C++ Reference**: cppreference.com
- **C++ Documentation**: cplusplus.com
- **Competitive Programming**: LeetCode, HackerRank, Codeforces
- **Books**: "The C++ Programming Language" (Bjarne Stroustrup)
- **YouTube**: Excellent C++ tutorial channels

Key Takeaway: C++ rewards careful programming. Understand memory management, use const correctness, and always consider performance. With these foundations, you'll be able to write efficient, powerful applications.

Happy coding in C++! 🚀